

Übersetzung von Vorlesungsunterlagen in TypeScript TheoInf I + II

Projektarbeit

des Studiengangs Angewandte Informatik
an der Dualen Hochschule Baden-Württemberg Mannheim

von
Tobias Neithöfer und Leo Vatter

Abgabedatum: 15. April 2026

Bearbeitungszeitraum

01.10.2025 - 15.04.2026

Kurs

TINF23AI2

Gutachter

Prof. Dr. Karl Stroetmann

Unterschrift Gutachter

Erklärung

gemäß § 35 (1) der „Studien- und Prüfungsordnung DHBW Technik“ vom 7. März 2024.

Wir haben die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet.

Wir versichern zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.*

* falls beide Fassungen gefordert sind

Mannheim, 23. Februar 2026

Ort, Datum

Unterschrift

Abstrakt

Kurze Zusammenfassung der Arbeit.

Abstract

Short summary of the paper.

Inhaltsverzeichnis

Abbildungsverzeichnis	VI
Abkürzungsverzeichnis	IX
1 Einleitung	1
1.1 Problemstellung	1
1.2 Zielsetzung	2
2 Grundlagen	4
2.1 Anaconda	4
2.2 Interaktive Programmierung mit Jupyter	5
2.3 TypeScript im Data-Science-Kontext	6
2.4 Setup und Toolchain	11
3 Einführung in TypeScript	14
3.1 Primitive Datentypen und Arithmetik	15
3.2 Ausgabe und Zeichenkettenlitterale	16
3.3 Arithmetische Operationen und Variablenzuweisung	18
3.4 Arrays	19
3.5 Tupel	25
3.6 Objekte	26
3.7 Boolesche Werte und Operatoren	27
3.8 Kontrollstrukturen	29
3.9 Numerische Funktionen	31
4 Methodische Vorgehensweise	33
4.1 Projektmanagement mit GitHub Projects	33
4.2 Der Übersetzungs-Workflow	34
4.3 Einsatz von KI-gestützten Tools	36
5 Case Studies	40
5.1 CNF	40
5.2 Pattern Matching und Type Guards	45

5.3	CNF-Konstruktion und Set-Operationen	47
5.4	Praktische Auswirkungen für Davis-Putnam	49
5.5	Zusammenfassung der Kernänderungen	50
5.6	Graphviz	51
5.7	Wiederverwendung von Notebooks	51
5.8	Recursive Set	51
6	Evaluierung und Ergebnisse	52
6.1	Laufzeit	52
6.2	Lesbarkeit	52
6.3	Effizienz	52
7	Fazit und Ausblick	53
	Literaturverzeichnis	X

Abbildungsverzeichnis

1	GitHub Projects Kanban-Board	33
---	--	----

List of Listings

1	Dynamische Typisierung in Python	8
2	Dynamische Typisierung in Python	9
3	Statische Typisierung in TypeScript	10
4	Typhinweise in Python mit MyPy-Prüfung	10
5	Bereitstellung der Laufzeitumgebung	12
6	Integration des TypeScript-Kernels	12
7	Installation der Projekt-Abhängigkeiten	13
8	Typüberprüfung numerischer Literale	15
9	Demonstration des Präzisionsverlusts bei Number	15
10	Verwendung von bigint	16
11	Ausgabe und Definition von Strings	17
12	Verwendung von Template Strings	17
13	Grundlegende arithmetische Operatoren	18
14	Ganzzahldivision und Modulo	18
15	Konstanten vs. Variablen	19
16	Definition eines leeren Arrays	19
17	Array mit numerischen Elementen	19
18	Zugriff auf Array-Elemente	20
19	Modifikation eines Array-Elements	20
20	Verkettung mit concat	20
21	Verkettung mit dem Spread-Operator	20
22	Bestimmung der Array-Länge	21
23	Signatur <i>filter</i>	21
24	Filtern gerader Zahlen	22
25	Signatur from	22
26	Erzeugung einer Zahlenfolge	23
27	Quadrieren von Zahlen mit map	23
28	Signatur from	24
29	Summierung mit reduce	24
30	Einfaches Array Destructuring	25
31	Destructuring mit Rest-Operator	25

32	Definition und Initialisierung eines Tupels	25
33	Zugriff auf Tupel-Elemente	26
34	Objektdefinition	26
35	Zugriffsarten auf Objekteigenschaften	26
36	Modifikation von Eigenschaften	27
37	Verschachtelte Objekte	27
38	Zugriff auf verschachtelte Daten	27
39	Logische Verknüpfungen	28
40	If-Else Verzweigung	29
41	Switch-Statement als Alternative	30
42	Optimiertes Sieb des Eratosthenes	31
43	Python: Generische rekursive Formeldefinition	40
44	TypeScript: Strukturell präzise Formeldefinition	41
45	Progressive Typdefinitionen für Transformationsschritte	42
46	NNF-Typ mit eingeschränkter Negation	42
47	Python: Verwendung von frozenset für Klauseln	43
48	Problem der Referenzgleichheit in JavaScript	43
49	TypeScript: RecursiveSet mit struktureller Gleichheit	44
50	Python: Literal-Repräsentation	44
51	TypeScript: Literal-Repräsentation mit Tuple	45
52	Triviality-Prüfung mit struktureller Gleichheit	45
53	Python: Pattern Matching mit match-Statement	46
54	TypeScript: Switch-Statement mit Type Guards	46
55	Python: CNF-Konstruktion mit Set Comprehensions	47
56	TypeScript: CNF-Konstruktion mit expliziten Schleifen	48
57	Duplikateliminierung in der Praxis	50
58	TypeScript: Beispiel mit Interface und Funktion	53

Abkürzungsverzeichnis

KI	Künstliche Intelligenz
JSON	JavaScript Object Notation

1 Einleitung

Die vorliegende Studienarbeit dokumentiert die systematische Portierung von Jupyter-Notebooks aus der Programmiersprache Python nach TypeScript. Diese Notebooks bilden die Grundlage für die Lehrveranstaltungen „Theoretische Informatik I“ (Logik) und „Theoretische Informatik II“ (Algorithmen und Datenstrukturen) und dienen als interaktive Vorlesungsunterlagen zur Vermittlung theoretischer Konzepte der Informatik.

Die Verwendung von Jupyter-Notebooks in dem Informatikstudium hat sich in den letzten Jahren als effektive Methode etabliert, um theoretische Konzepte mit praktischen Implementierungen zu verbinden [3]. Während Python aufgrund seiner Verbreitung und einfachen Syntax häufig als erste Programmiersprache gewählt wird, stellt die dynamische Typisierung in komplexeren Projekten zunehmend eine Herausforderung dar. Diese Problematik führt zur zentralen Fragestellung dieser Arbeit: Lässt sich durch den Einsatz statischer Typisierung die Codequalität, Nachvollziehbarkeit und Effizienz in Lehrveranstaltungen verbessern, ohne dabei die didaktische Zugänglichkeit zu beeinträchtigen?

1.1 Problemstellung

Die aktuelle Implementierung der Vorlesungsunterlagen basiert auf 80 Jupyter-Notebooks. Prof. Dr. Karl Stroetmann formulierte die Aufgabenstellung wie folgt:

„Meine Vorlesungen Theoretische Informatik I+II [2] [1] verwenden eine Reihe von Jupyter-Notebooks, die auf der Sprache Python basieren. Die Sprache Python ist nur dynamisch getypt. Zwar gibt es für Python den Typ-Checker MyPy, aber das von MyPy unterstützte Typsystem ist wesentlich schwerfälliger als das Typsystem von TypeScript. Mein Ziel ist es daher, meine Vorlesungen von Python auf TypeScript umzustellen.“

Die zentrale Problematik ergibt sich aus mehreren Aspekten:

Mangelnde Typsicherheit in der informatischen Ausbildung

Python ist eine dynamisch typisierte Sprache, bei der Typfehler erst zur Laufzeit erkannt werden. Dies führt insbesondere in der Lehre zu Herausforderungen, da Studierende häufig erst spät, nach Ausführung des Codes, auf Typinkompatibilitäten stoßen. Während Tools wie MyPy eine nachträgliche statische Typprüfung

ermöglichen, bleibt die Integration in bestehenden Python-Projekte oft komplex und das Typsystem weniger ausdrucksstark als bei nativ statisch typisierten Sprachen [7]. Diese verzögerte Fehlerkennung kann das Verständnis für Typsysteme und deren Bedeutung in der Softwareentwicklung beeinträchtigen.

Eingeschränkte Möglichkeiten der statischen Codeanalyse

TypeScript bietet als Superset von JavaScript ein robustes, strukturelles Typsystem, das bereits während der Entwicklung umfassende Fehlerprüfungen ermöglicht [7]. Die statische Typprüfung zur Compile-Zeit erlaubt es, viele potenzielle Laufzeitfehler bereits im Vorfeld zu identifizieren und trägt somit zu einer verbesserten Code-Qualität bei. In der Lehre kann dies Studierenden ein tieferes Verständnis für Datenstrukturen und Algorithmen vermitteln, da Typen explizit deklariert und durch den Compiler überprüft werden.

Umfang des Portierungsprojekts

Mit insgesamt etwa 80 Notebooks stellt der Umfang der zu übersetzenden Materialien eine erhebliche Herausforderung dar. Diese Menge erfordert eine systematische Vorgehensweise sowie den strategischen Einsatz von KI-gestützten Übersetzungstools, um Konsistenz und Qualität über alle Notebooks hinweg zu gewährleisten.

1.2 Zielsetzung

Die vorliegende Arbeit verfolgt mehrere miteinander verbundene Zielsetzungen. Das vom Dozenten vorgegebene Hauptziel besteht in der vollständigen Übersetzung der 80 Python-basierten Jupyter-Notebooks nach TypeScript unter Verwendung von Künstlicher Intelligenz (KI)-Tools wie ChatGPT, Gemini, DeepSeek oder Grok. Diese Portierung soll die Vorlesungsmaterialien auf eine Sprache mit statischer Typisierung überführen und damit die didaktischen Möglichkeiten erweitern.

Ein zentrales Forschungsziel liegt in der Untersuchung, ob die in TypeScript implementierten Programme eine vergleichbare oder möglicherweise verbesserte Verständlichkeit aufweisen als die ursprünglichen Python-Implementierungen. Diese Fragestellung ist von besonderer Relevanz für die Lehrpraxis, da die Zugänglichkeit des Codes für Studierende nicht durch die erhöhte Typsicherheit beeinträchtigt werden sollte.

Zusätzlich zur qualitativen Bewertung soll anhand der übersetzten Notebooks ein quantitativer Performanzvergleich zwischen Python und TypeScript durchgeführt werden. Dieser Vergleich kann Aufschluss über die praktische Eignung beider Sprachen für rechenintensive Algorithmen und Datenstrukturen geben. Die Arbeit folgt dabei einem systematischen Ansatz, der den Einsatz von KI-gestützten Übersetzungstools mit manueller Überprüfung und Optimierung kombiniert, wobei Best Practices für die TypeScript-Entwicklung berücksichtigt und die Übersetzungsqualität kontinuierlich überprüft werden.

2 Grundlagen

Dieses Kapitel erläutert die technologischen Grundlagen und Werkzeuge, die für die Durchführung dieser Studienarbeit relevant sind. Zunächst wird die Distributionsplattform Anaconda vorgestellt, gefolgt von einer detaillierten Betrachtung der interaktiven Entwicklungsumgebung Jupyter. Anschließend erfolgt eine Einführung in die Zielsprache TypeScript sowie der Vergleich mit Python. Den Abschluss bildet die Beschreibung der spezifischen Setup- und Toolchain-Konfiguration.

2.1 Anaconda

Anaconda ist eine weit verbreitete Open-Source-Distribution für die Programmiersprachen Python und R, die speziell für Aufgaben in den Bereichen Data Science, maschinelles Lernen und wissenschaftliches Rechnen entwickelt wurde. Sie vereinfacht das Paketmanagement und die Bereitstellung von Softwareumgebungen erheblich, was insbesondere in komplexen wissenschaftlichen Projekten von Vorteil ist. Kernkomponente der Distribution ist der Paketmanager **conda**. Dieser ermöglicht es, Softwarepakete und deren Abhängigkeiten plattformübergreifend zu installieren, zu aktualisieren und zu verwalten. Ein wesentliches Merkmal von Anaconda ist die Fähigkeit, isolierte Umgebungen (Environments) zu erstellen. Diese Isolation verhindert Konflikte zwischen verschiedenen Versionen von Bibliotheken, die für unterschiedliche Projekte benötigt werden könnten.

Für die vorliegende Arbeit dient Anaconda als primäre Basisinfrastruktur. Obwohl das Ziel der Portierung auf TypeScript liegt, wird Anaconda ebenfalls verwendet, um die ursprünglichen Python-Notebooks auszuführen und zu validieren. Darüber hinaus fungiert das Anaconda Prompt (bzw. das Terminal) als Schnittstelle für systemweite Befehle. Hierüber erfolgen auch die notwendigen Installationen für das TypeScript-Ökosystem, wie beispielsweise die Ausführung von `npm install`, um Node.js-Pakete global oder lokal verfügbar zu machen. Die Verwaltung der Node.js-Laufzeitumgebung kann ebenfalls in die bestehende Infrastruktur integriert werden, was Anaconda zu einem vielseitigen Werkzeug macht.

2.2 Interaktive Programmierung mit Jupyter

Das Jupyter Notebook ist eine webbasierte, interaktive Entwicklungsumgebung, die es ermöglicht, Dokumente zu erstellen, die sowohl ausführbaren Code als auch formatierten Text, mathematische Formeln und Visualisierungen enthalten. Der Name „Jupyter“ ist ein Akronym, das sich ursprünglich aus den Kernsprachen Julia, Python und R ableitete, mittlerweile aber eine Vielzahl weiterer Programmiersprachen unterstützt.

Die Architektur von Jupyter basiert auf einem Client-Server-Modell, das die Trennung zwischen der Benutzeroberfläche und der Code-Ausführung ermöglicht.

- Das Frontend (Client): Der Benutzer interagiert über einen Webbrowser mit dem Notebook. Die Oberfläche erlaubt die Eingabe von Code in Zellen sowie die Gestaltung von Textbereichen mittels Markdown.
- Der Notebook-Server: Dieser Server verwaltet die Dateien und vermittelt die Kommunikation zwischen dem Frontend und dem ausführenden Kernel.
- Das Dateiformat (.ipynb): Jupyter Notebooks werden als JavaScript Object Notation ([JSON](#))-Dateien mit der Endung `.ipynb` gespeichert. Diese strukturierte Textdatei enthält den gesamten Inhalt des Notebooks, Code, Markdown-Text, Metadaten sowie die Ausgabeartefakte (z. B. Konsolenausgaben oder Grafiken), in einem standardisierten Format. Dies erleichtert die Versionierung und den Austausch von wissenschaftlichen Ergebnissen.

Ein zentrales Element der Jupyter-Architektur ist der Kernel. Der Kernel ist ein separater Prozess, der für die Ausführung des Codes zuständig ist, den der Benutzer im Notebook eingibt. Wenn eine Code-Zelle ausgeführt wird, sendet das Frontend den Code an den Kernel. Dieser führt die Anweisungen aus und sendet das Ergebnis zurück an das Frontend zur Darstellung. Für diese Studienarbeit ist das Kernel-Konzept von entscheidender Bedeutung:

Während die ursprünglichen Vorlesungsunterlagen auf dem Standard-Python-Kernel (ipykernel) basieren, erfordert die Portierung auf TypeScript die Integration eines dedizierten TypeScript-Kernels (tslab). Diese Architektur ermöglicht den Wechsel der Programmiersprache innerhalb derselben Benutzeroberfläche durch bloßen Austausch des Backends.

Die technische Integration des TypeScript-Kernels wird in Abschnitt [Setup und Toolchain](#) detailliert beschrieben.

2.3 TypeScript im Data-Science-Kontext

Während Python als De-facto-Standard im Bereich Data Science gilt, gewinnt das TypeScript-Ökosystem zunehmend an Bedeutung, insbesondere wenn Modelle direkt in Webanwendungen integriert oder visualisiert werden sollen.

2.3.1 Herausforderungen und Bibliotheks-Ökosystem

Eine wesentliche Herausforderung bei der Migration von Python zu TypeScript ist die Diskrepanz in der Standardbibliothek. Während Python mit Modulen wie `heapq` oder einer nativen Unterstützung für komplexe Mengenoperationen und Tupel ausgestattet ist, fehlen diese Konstrukte in der Standard-Laufzeitumgebung von TypeScript oft oder sind für wissenschaftliche Zwecke nicht performant genug implementiert. Zudem existiert zwar ein reiches Ökosystem (npm), dieses ist jedoch stark fragmentiert, im Gegensatz zu den monolithischen Standard-Paketen in Python wie beispielsweise NumPy.

Um die Funktionalität der Python-Vorlesungsunterlagen zu replizieren, wird in dieser Arbeit auf eine Auswahl spezialisierter Bibliotheken zurückgegriffen:

Mathematik und Logik:

Für numerische Stabilität und symbolische Berechnungen kommen `mathjs` (umfassende Mathematik-Bibliothek) und `fraction.js` (für rationale Zahlen zur Vermeidung von Fließkommafehlern) zum Einsatz. Für Constraint-Solving-Probleme und logische Beweise werden der `logic-solver` sowie der `z3-solver` (ein Theorem-Prover von Microsoft Research) eingebunden.

Datenstrukturen:

Da TypeScript keine native Priority Queue besitzt (äquivalent zu Pythons `heapq`), wird `heap-js` verwendet. Eine Besonderheit stellt die Bibliothek `recursive-set` dar. Native JavaScript `Sets` prüfen Objekte auf Referenzgleichheit, nicht auf Wertegleichheit. Das bedeutet, zwei inhaltlich identische Objekte werden als unterschiedlich behandelt. `recursive-set` ermöglicht echte Mengenoperationen auf Basis der tiefen Gleichheit (Deep Equality).

Visualisierung und Medien:

Zur Generierung von Graphen wird `@hpc-js/wasm` genutzt. Dies ist eine WebAssembly-Portierung von Graphviz, die das Rendern von DOT-Sprache ermöglicht, ohne dass eine lokale `graphviz.exe`-Installation auf dem Host-System notwendig ist – ein entscheidender Vorteil für die Portabilität des Notebooks. Für die Bildmanipulation wird `pngjs` verwendet.

Type Definitions:

Ein Spezifikum der TypeScript-Entwicklung ist die Notwendigkeit separater Typ-Definitionen für Bibliotheken, die ursprünglich in reinem JavaScript geschrieben wurden. So muss beispielsweise für die Bildverarbeitung zusätzlich das Paket `@types/pngjs` als *Dev-Dependency* installiert werden, um die statische Typprüfung zu ermöglichen.

@TobiKnight

2.3.2 Syntaxvergleich und Typisierungskonzepte

Der fundamentale Unterschied zwischen Python und TypeScript liegt im Zeitpunkt und der Strenge der Typprüfung.

Statisch vs. Dynamisch / MyPy vs. TypeScript Compiler

Um diesen Unterschied zu verstehen, ist es hilfreich, zunächst zu klären, was ein *Datentyp* ist. Jeder Wert, den ein Programm verarbeitet, hat eine bestimmte Art: Eine ganze Zahl wie 42 verhält sich anders als ein Text wie "Hallo" oder ein Wahrheitswert wie `True`. Diese verschiedenen Arten von Werten nennt man *Datentypen*. Der grundlegende Unterschied zwischen Python und TypeScript liegt darin, *wann* und *ob überhaupt* geprüft wird, ob diese Typen im Programm korrekt verwendet werden.

Dynamische Typisierung in Python

Python verwendet standardmäßig *dynamische Typisierung*. Das bedeutet, dass der Typ einer Variable nicht im Voraus festgelegt werden muss. Python ermittelt den Typ erst dann, wenn das Programm tatsächlich ausgeführt wird, also zur sogenannten *Laufzeit*. Als Analogie lässt sich ein Rucksack vorstellen, in den man beliebige Dinge (Datentypen) packen kann: Mal liegt ein Buch darin, mal eine Flasche Wasser. Erst wenn man etwas

herausnehmen will und feststellt, dass es nicht passt, gibt es ein Problem.

In Python ist es daher ohne Weiteres möglich, einer Variable zunächst eine Zahl und später einen Text zuzuweisen:

```
1 x = 42          # x hat den Typ int (ganze Zahl)
2 x = "Hallo"     # x hat jetzt den Typ str (Text) Python erlaubt dies
3 # Der Fehler tritt erst beim Ausführen der nächsten Zeile auf:
4 x + 1
```

Listing 1: Dynamische Typisierung in Python

Den Fehler in der letzten Zeile würde Python erst bemerken, wenn diese Zeile tatsächlich ausgeführt wird. In einem großen Programm mit vielen Zeilen Code kann das dazu führen, dass Fehler lange unentdeckt bleiben, zum Beispiel dann, wenn eine bestimmte Stelle im Code nur selten erreicht wird.

2.3.3 Syntaxvergleich und Typisierungskonzepte

Der fundamentale Unterschied zwischen Python und TypeScript liegt im Zeitpunkt und der Strenge der Typprüfung.

Um diesen Unterschied zu verstehen, ist es hilfreich, zunächst zu klären, was eine *Variable* und was ein *Datentyp* ist. In der Programmierung speichert man Daten in Variablen. Man kann sich eine Variable wie eine Kiste vorstellen: Der *Name* der Variable (z. B. `x`) steht als Etikett außen auf der Kiste, und der eigentliche *Wert* (z. B. `42`) liegt darin. Jeder dieser Werte hat zudem eine bestimmte Art: Eine ganze Zahl wie `42` verhält sich anders als ein Text wie `"Hallo"` oder ein Wahrheitswert wie `True`. Diese verschiedenen Arten von Werten nennt man *Datentypen*.

Der grundlegende Unterschied zwischen Python und TypeScript liegt darin, *wann* und *ob überhaupt* geprüft wird, ob diese Typen im Programm korrekt verwendet werden.

Dynamische Typisierung in Python

Python verwendet standardmäßig *dynamische Typisierung*. Das bedeutet, dass der Typ einer Variable nicht im Voraus festgelegt werden muss. Python ermittelt den Typ erst dann, wenn das Programm tatsächlich ausgeführt wird, also zur sogenannten *Laufzeit*. Als Analogie lässt sich eine unbeschriftete Schublade vorstellen. Zuerst legt man einen

Hammer (eine Zahl) hinein. Später räumt man die Schublade aus und legt stattdessen einen Pinsel (einen Text) hinein, die Schublade akzeptiert beides problemlos. Der Fehler fällt erst in dem Moment auf, in dem man blind in die Schublade greift, um einen Nagel in die Wand zu schlagen: Man merkt erst beim Zugriff auf die Schublade (Variable), dass man mit einem Pinsel nicht hämmern kann.

In Python ist es daher ohne Weiteres möglich, einer Variable zunächst eine Zahl und später einen Text zuzuweisen:

```
1 x = 42          # x hat den Typ int (ganze Zahl)
2 x = "Hallo"     # x hat jetzt den Typ str (Text) -- Python erlaubt dies
3 # Der Fehler tritt erst beim Ausführen der naechsten Zeile auf:
4 x + 1
```

Listing 2: Dynamische Typisierung in Python

Den Fehler in der letzten Zeile würde Python erst bemerken, wenn diese Zeile tatsächlich ausgeführt wird. In einem großen Programm mit vielen Zeilen Code kann das dazu führen, dass Fehler lange unentdeckt bleiben, zum Beispiel dann, wenn eine bestimmte Stelle im Code nur selten erreicht wird.

Statische Typisierung in TypeScript

TypeScript hingegen verwendet *statische Typisierung*. Der Typ einer Variable muss entweder explizit angegeben werden oder wird vom Compiler automatisch erkannt. Diese automatische Erkennung nennt man *Typinferenz* (abgeleitet von *inferieren*, also schlussfolgern). Entscheidend ist, dass TypeScript alle Typen bereits *vor* der Ausführung des Programms überprüft. Dieser Vorgang heißt *Kompilierung*, und das Programm, das diese Prüfung durchführt, nennt man *TypeScript-Compiler*.

Der TypeScript-Compiler liest den gesamten Code durch und prüft, ob alle Typangaben widerspruchsfrei sind. Findet er einen Fehler, bricht er ab und gibt eine Fehlermeldung aus. Das Programm wird also gar nicht erst gestartet.

Das folgende Beispiel zeigt, wie TypeScript denselben Fehler verhindert. Eine Methode um in TypeScript eine neue Variable anzulegen, ist die Verwendung des Schlüsselworts `let`. Es signalisiert dem Compiler: Hier wird eine neue Variable erstellt. Dahinter folgt der Name der Variable, ein Doppelpunkt und der gewünschte Datentyp gefolgt von einem

Gleichzeichen gefolgt von dem zuzuweisenden Wert.

```
1 let x: number = 42;
2 // Die folgende Zeile wuerde vom Compiler sofort rot markiert werden:
3 x = "Hallo";
4 // Fehlermeldung: Type 'string' is not assignable to type 'number'.
```

Listing 3: Statische Typisierung in TypeScript

Der Compiler würde bei der Zuweisung des Textes sofort einen Fehler melden, da in eine `number`-Variable kein `string` (Text) gelegt werden darf. Der Vorteil ist, dass solche Fehler bereits beim Schreiben des Codes und nicht erst beim Ausführen gefunden werden.

MyPy als optionale Typprüfung für Python

Da die dynamische Typisierung in größeren Projekten fehleranfällig sein kann, wurde für Python das Werkzeug *MyPy* entwickelt. MyPy ist ein *statischer Typprüfer* für Python: Es analysiert den Code auf Typfehler, indem es ihn lediglich durchliest, ohne das Programm tatsächlich zu starten. Anders als der TypeScript-Compiler, der den Code nach der Prüfung auch übersetzt, ist MyPy ein reines Kontrollwerkzeug und belässt den Python-Code unverändert.

Ein wesentlicher Unterschied zum TypeScript-Compiler ist, dass MyPy vollständig *optional* ist. Um MyPy nutzen zu können, müssen im Code freiwillige Typangaben vorhanden sein, sogenannte *Type Hints* (Typhinweise). Diese Typhinweise haben keinen Einfluss auf das Verhalten des Programms, Python ignoriert sie bei der Ausführung vollständig.

Das folgende Beispiel verdeutlicht dies mit einer einfachen Variable:

```
1 # Die Variable wird mit dem Typhinweis ': int' (ganze Zahl) versehen
2 alter: int = 20
3 # MyPy meldet hier VOR dem Start einen Fehler, da "Zwanzig" ein Text ist.
4 alter = "Zwanzig"
```

Listing 4: Typhinweise in Python mit MyPy-Prüfung

Bedeutung für die Übersetzung

Für die Übersetzung von den Python-Notebooks in TypeScript-Notebooks ergibt sich aus dem Unterschied der Typprüfung eine wichtige Herausforderung. Die Python-Notebooks können in drei Zuständen vorliegen:

1. **Vollständig ungetypter Code:** Es sind keine Typangaben vorhanden. Die Typen müssen also beim Übersetzen ermittelt und ergänzt werden.
2. **Teilweise getypter Code:** Nur manche Stellen enthalten Typhinweise. Hier müssen einerseits die fehlenden Typen für TypeScript ermittelt und ergänzt werden. Andererseits können die bereits vorhandenen Typangaben oft nicht einfach eins zu eins übernommen werden, sondern erfordern ebenfalls Anpassungen.
3. **Vollständig mit MyPy annotierter Code:** Alle relevanten Stellen enthalten Typhinweise. Diese können zwar oft direkt in TypeScript-Typen übertragen werden, jedoch müssen selbst hier häufig noch Präzisierungen vorgenommen werden. Der Grund dafür ist, dass das Typsystem von TypeScript in manchen Bereichen anders strukturiert oder strenger ist als das von MyPy.

2.4 Setup und Toolchain

Die Reproduzierbarkeit der Ergebnisse und die Stabilität der Entwicklungsumgebung sind essenziell für die Validierung der portierten Artefakte. Dieses Unterkapitel beschreibt die Architektur der verwendeten Toolchain und dokumentiert die notwendigen Konfigurationsschritte, wie sie in der [README.md](#) der Arbeit definiert sind.

2.4.1 Architektur der Laufzeitumgebung

Die technische Infrastruktur basiert auf einer hybriden Architektur, die das Python-zentrierte Ökosystem von Anaconda mit der JavaScript-basierten Node.js-Laufzeitumgebung verbindet. Diese Trennung ist notwendig, da Jupyter ursprünglich für Python konzipiert wurde, TypeScript jedoch eine Node.js-Laufzeit für die Transpilierung und Ausführung benötigt.

Die Toolchain gliedert sich in drei Schichten:

1. **Basisinfrastruktur (Anaconda):** Bereitstellung von Python, Jupyter (nbclassic) und Verwaltung der Systempfade.

2. **Laufzeitumgebung (Node.js):** Ausführungsumgebung für den TypeScript-Compiler und die genutzten Bibliotheken.
3. **Kernel-Layer (tslab):** Eine Middleware, die das Jupyter-Messaging-Protokoll implementiert und TypeScript-Codezellen an die Node.js-Laufzeit weiterleitet.

2.4.2 Implementierung der Umgebung

Die Einrichtung der Umgebung erfolgt deklarativ über die Kommandozeile der Anaconda-Distribution. Zunächst wird die Laufzeitumgebung durch die Installation von Node.js und dem klassischen Jupyter-Frontend vorbereitet. Dies schafft die Voraussetzungen für die Integration nicht-nativer Kernel.

```
1      conda activate base
2      conda install nodejs
3      conda install -c conda-forge nbclassic
```

Listing 5: Bereitstellung der Laufzeitumgebung

Für die Ausführung von TypeScript innerhalb der Notebooks wird der Kernel **tslab** verwendet. Im Gegensatz zu reinen JavaScript-Kerneln (wie IJavascript) ermöglicht **tslab** eine direkte Typ-Prüfung innerhalb der Zellen, was für die didaktische Zielsetzung der Arbeit, die Vermittlung von Typsicherheit, unabdingbar ist.

```
1      tslab install
2      npm install -g tslab
3      npm install typescript
```

Listing 6: Integration des TypeScript-Kernels

Die fachlichen Anforderungen der Vorlesungen erfordern spezifische Bibliotheken, die über den Paketmanager **npm** bezogen werden. Hierbei wird strikt zwischen Laufzeit-Abhängigkeiten (z.B. **mathjs** für Numerik, **recursive-set** für Mengenlehre) und Entwicklungs-Abhängigkeiten (z.B. **@types/pngjs**) unterschieden. Letztere sind notwendig, um für ursprünglich in JavaScript geschriebene Bibliotheken eine statische Typprüfung zu ermöglichen.

```
1 // Mathematische und logische Kernkomponenten
2 npm install fraction.js mathjs logic-solver z3-solver
3
4 // Datenstrukturen und Mengenoperationen
5 npm install heap-js recursive-set@8.0.0
6
7 // Visualisierung und Medienverarbeitung
8 npm install @hpcc-js/wasm pngjs
9 npm i --save-dev @types/pngjs
```

Listing 7: Installation der Projekt-Abhängigkeiten

2.4.3 Validierung der Toolchain

Um sicherzustellen, dass die Interaktion zwischen dem Jupyter-Frontend, dem tslab-Kernel und den installierten Node-Modulen korrekt funktioniert, wurde ein dediziertes Testverfahren etabliert. Das Referenz-Notebook [Test-Libraries.ipynb](#) führt einen systematischen Import-Test aller Komponenten durch. Dieses Notebook verifiziert, dass:

- der Kernel korrekt registriert ist und TypeScript-Syntax verarbeitet,
- alle externen Module im Pfad der Laufzeitumgebung gefunden werden,
- die Typdefinitionen korrekt aufgelöst werden.

Erst nach erfolgreicher Ausführung dieses Notebooks gilt die Umgebung als valide für die Bearbeitung der Vorlesungsunterlagen.

3 Einführung in TypeScript

TypeScript wurde von Microsoft entwickelt und erstmals im Oktober 2012 der Öffentlichkeit vorgestellt. Die Entwicklung stand unter der Leitung von Anders Hejlsberg, einem renommierten Softwarearchitekten, der zuvor bereits maßgeblich an der Gestaltung von C#, Delphi und Turbo Pascal beteiligt war. [4] [6] Der primäre Beweggrund für die Schaffung dieser neuen Sprache lag in den Defiziten von JavaScript bei der Entwicklung großer, komplexer Anwendungen [4].

Als Antwort auf diese Herausforderungen ist TypeScript als eine Erweiterung von JavaScript konzipiert, mit dem expliziten Ziel, die Entwicklung großer und langfristig wartbarer Anwendungen zu erleichtern. Syntaktisch ist TypeScript als Superset von ECMAScript 5 definiert, sodass jedes gültige JavaScript-Programm zugleich ein gültiges TypeScript-Programm ist. Zusätzlich zu JavaScript ergänzt TypeScript unter anderem Interfaces sowie ein reichhaltiges graduelles Typsystem, das eine schrittweise (inkrementelle) Typisierung bestehender Codebasen unterstützt. [7]

Ein wesentliches Entwurfsprinzip ist dabei die *Typauslöschung* (type erasure): Die Typinformationen werden vom Compiler zur statischen Analyse genutzt, sind jedoch nicht Bestandteil des erzeugten JavaScript-Codes, sodass zur Laufzeit keine TypeScript-Typen repräsentiert oder automatisch geprüft werden. Der TypeScript-Compiler überprüft TypeScript-Programme lediglich zur Übersetzungszeit und transformiert sie in JavaScript, wodurch sie unmittelbar in etablierten JavaScript-Laufzeitumgebungen ausgeführt werden können. Bierman, Abadi und Torgersen (2014) beschreiben TypeScript zudem als bewusst nicht vollständig statisch typsicher ausgelegt, um gängige JavaScript-Idiome und die Typisierung existierender Bibliotheken praktisch zu unterstützen. [7]

Nach der Einführung in die Entstehungsgeschichte und die zentralen Entwurfsprinzipien von TypeScript soll im Folgenden die Programmiersprache selbst vorgestellt werden. Ziel ist es, einen Überblick über die grundlegenden Sprachkonstrukte zu geben, welche für die Vorlesungsunterlagen relevant sind.

Die Darstellung orientiert sich dabei am Notebook [Introduction.ipynb](#), das als Einstieg der Vorlesung *Theoretische Informatik* dient. Anhand dieses Notebooks werden zentrale Konzepte wie Variablendeklarationen, Typannotationen, Funktionen, Kontrollstrukturen, Arrays, Objekte sowie grundlegende Aspekte des Typsystems eingeführt und anhand von

Codebeispielen illustriert.

3.1 Primitive Datentypen und Arithmetik

In TypeScript, wie auch in JavaScript, werden alle numerischen Werte standardmäßig als Gleitkommazahlen gemäß dem [IEEE-754-Standard](#) repräsentiert. Es existiert kein separater Datentyp für Ganzzahlen (Integers). Dies gilt selbst dann, wenn die Darstellung eine Ganzzahl vermuten lässt. Der Operator `typeof` ist ein Operator, der zur Laufzeit den Typ seines Operanden als Zeichenkette zurückgibt und zur einfachen Typinspektion eingesetzt wird.

```
1  typeof 666    // Ausgabe: "number"
2  typeof 666.0 // Ausgabe: "number"
```

Listing 8: Typüberprüfung numerischer Literale

Diese Designentscheidung hat direkte Auswirkungen auf die Präzision arithmetischer Operationen. Die Genauigkeit für Ganzzahlen ist durch die Konstante `Number.MAX_SAFE_INTEGER` begrenzt, welche den Wert $2^{53} - 1$ (ca. 9 Milliarden) repräsentiert. Jenseits dieser Grenze ist die exakte Darstellung von Ganzzahlen nicht mehr gewährleistet, was zu Präzisionsverlusten führt.

```
1  Number.MAX_SAFE_INTEGER + 1 == Number.MAX_SAFE_INTEGER + 2
2  // Ausgabe: true
```

Listing 9: Demonstration des Präzisionsverlusts bei Number

Um die Limitierungen des `Number`-Typs zu umgehen, existiert der primitive Datentyp `bigint`. Er ermöglicht die Darstellung von Ganzzahlen beliebiger Größe, beschränkt lediglich durch den verfügbaren Arbeitsspeicher. Syntaktisch werden `bigint`-Literele durch das Suffix `n` gekennzeichnet. Ergänzend steht der Wrapper-Konstruktor `BigInt()` zur Verfügung, der einen gegebenen Wert in den primitiven Typ `bigint` konvertiert – analog zur Beziehung zwischen `number` und dem Wrapper-Objekt `Number`.

Ein wichtiger Aspekt der Typsicherheit in TypeScript ist die strikte Trennung dieser beiden numerischen Typen. Es ist nicht möglich, `bigint` und `number` in arithmetischen

Operationen ohne explizite Konvertierung zu mischen. Zudem unterscheidet sich das Verhalten bei der Division: Während **number** stets Gleitkommaergebnisse liefert, führt die Division von **bigint**-Werten zu einer Trunkierung des Dezimalanteils (Ganzzahldivision).

Für die Deklaration unveränderlicher Variablen wird in TypeScript das Schlüsselwort **const** verwendet. Eine mit **const** deklarierte Variable kann nach ihrer initialen Zuweisung nicht neu zugewiesen werden, was die Absicht der Unveränderlichkeit explizit im Code ausdrückt.

```
1  const result: bigint = 7n ** 125n;  
2  // Berechnung extrem großer Potenzen  
3  
4  5n / 2n  
5  // Ausgabe: 2n (Nachkommastellen werden abgeschnitten)
```

Listing 10: Verwendung von **bigint**

3.2 Ausgabe und Zeichenkettenlitterale

Für die Ausgabe von Werten auf der Konsole stellt TypeScript die Funktion `console.log` bereit. Diese Funktion ist polymorph und akzeptiert Argumente beliebigen Typs, um sie in einer textuellen Repräsentation auszugeben. Ein elementarer Datentyp für textuelle Daten ist der **String**. In TypeScript können Zeichenkettenlitterale äquivalent durch einfache Anführungszeichen (') oder doppelte Anführungszeichen (") definiert werden. Es gibt semantisch keinen Unterschied zwischen diesen beiden Notationen, was Entwicklern Flexibilität bei der Einhaltung von Coding-Style-Guidelines gewährt. Zusätzlich existiert eine dritte Form von Zeichenkettenlitteralen, die durch Backticks (`) gekennzeichnet ist und als Grundlage für sogenannte Template Strings dient, auf die im Folgenden gesondert eingegangen wird.

In der interaktiven Umgebung eines Jupyter Notebooks wird zudem das Ergebnis des letzten Ausdrucks einer Zelle automatisch ausgegeben, ohne dass ein expliziter Aufruf von `console.log` notwendig ist.

```
1 // Explizite Ausgabe über die Konsole
2 console.log('Hello, World!');
3
4 // Implizite Ausgabe des letzten Ausdrucks im Notebook
5 'Hello, World!'
6
7 // Alternative Syntax mit doppelten Anführungszeichen
8 "Hello, World!"
```

Listing 11: Ausgabe und Definition von Strings

Template String

Zusätzlich zu den klassischen Zeichenkettenliteralen unterstützt TypeScript sogenannte *Template Strings*. Diese werden durch Backticks (``) umschlossen und bieten erweiterte Funktionalitäten gegenüber herkömmlichen Strings. Ein zentrales Merkmal ist die String-Interpolation, die es ermöglicht, Ausdrücke direkt in eine Zeichenkette einzubetten. Hierbei wird die Syntax `\${...}` verwendet, wobei der Inhalt der geschweiften Klammern ausgewertet und das Ergebnis in den String konvertiert wird. Dies erhöht die Lesbarkeit des Codes erheblich, da komplexe Verkettungsoperationen vermieden werden. Zudem unterstützen Template Strings mehrzeilige Zeichenketten ohne spezielle Escape-Sequenzen.

```
1 const name = "Alice";
2 const age = 25;
3
4 // Direkte Einbettung von Variablen
5 console.log(`User ${name} is ${age} years old.`);
6 // Ausgabe: User Alice is 25 years old.
7
8 // Auswertung von Ausdrücken innerhalb des Strings
9 console.log(`Next year, she will be ${age + 1}.`);
10 // Ausgabe: Next year, she will be 26.
```

Listing 12: Verwendung von Template Strings

3.3 Arithmetische Operationen und Variablenzuweisung

TypeScript stellt, analog zu JavaScript, die üblichen arithmetischen Operatoren zur Verfügung. Hierzu zählen die Addition (+), Subtraktion (-), Multiplikation (*), Division (/) sowie der Exponentiationsoperator (**) und der Modulo-Operator (%), der den Rest einer Division bestimmt.

```
1 1 + 2;    // Addition:      ergibt 3
2 5 - 3;    // Subtraktion:   ergibt 2
3 4 * 3;    // Multiplikation: ergibt 12
4 7 / 2;    // Division:      ergibt 3.5
5 7 % 3;    // Modulo:        ergibt 1
6 2 ** 10;  // Potenzierung:   ergibt 1024
```

Listing 13: Grundlegende arithmetische Operatoren

Ein wesentlicher Unterschied zu vielen anderen Programmiersprachen betrifft die Division. In TypeScript liefert der Standard-Divisionsoperator (/) stets eine Gleitkommazahl als Ergebnis, auch wenn beide Operanden Ganzzahlen sind. Um eine Ganzzahldivision (*Integer Division* oder Euklidische Division) durchzuführen, muss das Ergebnis explizit abgerundet werden. Hierfür steht die Funktion `Math.floor()` zur Verfügung, welche die größte Ganzzahl zurückgibt, die kleiner oder gleich dem übergebenen Argument ist. Der Modulo-Operator % berechnet den Rest dieser Division.

```
1 7 / 3;    // Standard-Division: ergibt 2.333...
2 Math.floor(7 / 3); // Ganzzahldivision: ergibt 2
3 7 % 3;    // Rest der Division: ergibt 1
```

Listing 14: Ganzzahldivision und Modulo

Zur Speicherung von Werten unterscheidet TypeScript zwischen Konstanten und Variablen. Das Schlüsselwort `const` deklariert eine Konstante, deren Wert nach der Initialisierung nicht mehr verändert werden kann (Reassigning ist verboten). Dies fördert die Unveränderlichkeit von Daten, wo immer es möglich ist. Soll sich ein Wert im Laufe des Programms ändern, muss die Variable mit dem Schlüsselwort `let` deklariert werden. Im Gegensatz zu `const` erlaubt `let` eine erneute Zuweisung des Variablenwertes, bleibt jedoch, anders als das ältere Schlüsselwort `var`, auf den umschließenden Block (*block scope*) beschränkt.

```

1 // Deklaration von Konstanten
2 const q = Math.floor(7 / 3);
3 const r = 7 % 3;
4 q = 5; // Error: Assignment to constant variable.
5
6 // Deklaration veränderlicher Variablen
7 let count = 1;
8 count = 2; // Erlaubt, da 'count' mit 'let' deklariert wurde

```

Listing 15: Konstanten vs. Variablen

3.4 Arrays

Arrays sind eine fundamentale Datenstruktur in TypeScript, die eine geordnete Sammlung von Elementen repräsentiert. Sie entsprechen konzeptionell den Listen in Python, unterscheiden sich jedoch in Syntax und bestimmten Verhaltensweisen. Ein Array wird in TypeScript durch eckige Klammern [und] definiert. Das leere Array, also ein Array ohne Elemente, wird wie folgt notiert:

```

1 []

```

Listing 16: Definition eines leeren Arrays

Ein Array mit konkreten Elementen wird durch Komma-separierte Werte innerhalb der eckigen Klammern erzeugt. Leerzeichen nach den Kommas sind optional, werden jedoch zur besseren Lesbarkeit empfohlen.

```

1 [1, 2, 3]

```

Listing 17: Array mit numerischen Elementen

Zugriff auf Array-Elemente: Der Index-Operator

Der *Index-Operator* `[.]` ist ein Postfix-Operator, der verwendet wird, um auf das Element an einer bestimmten Position (Index) zuzugreifen. TypeScript verwendet, wie die meisten modernen Programmiersprachen, eine nullbasierte Indizierung (*zero-based indexing*), d. h., das erste Element eines Arrays hat den Index 0.

```
1   const L = [1, 2, 3];  
2   L[0]; // Gibt das erste Element zurück: 1
```

Listing 18: Zugriff auf Array-Elemente

Der Index-Operator kann auch auf der linken Seite einer Zuweisung verwendet werden, um ein bestehendes Element zu modifizieren:

```
1   L[0] = 7;  
2   L; // Ergebnis: [7, 2, 3]
```

Listing 19: Modifikation eines Array-Elements

Verkettung von Arrays

Arrays können mittels der Methode `concat` verkettet werden. Diese Methode erstellt ein neues Array, das alle Elemente des aufrufenden Arrays gefolgt von allen Elementen des als Argument übergebenen Arrays enthält.

```
1   [1, 2, 3].concat([4, 5, 6]); // Ergebnis: [1, 2, 3, 4, 5, 6]
```

Listing 20: Verkettung mit `concat`

Alternativ kann der *Spread-Operator* `...` verwendet werden, um Arrays zu entpacken und zu kombinieren. Diese Syntax ist kompakter und wird in TypeScript häufig bevorzugt.

```
1   [...[1, 2, 3], ...[4, 5, 6]]; // Ergebnis: [1, 2, 3, 4, 5, 6]
```

Listing 21: Verkettung mit dem Spread-Operator

Die Eigenschaft `length` gibt die Anzahl der Elemente in einem Array zurück. Formal ist die Länge eines Arrays L definiert als $|L|$, die Kardinalität der Menge der enthaltenen Elemente.

```
1 [4, 5, 4].length; // Ergebnis: 3
```

Listing 22: Bestimmung der Array-Länge

Die Methode `filter`

Die Methode `filter` ist eine Higher-Order-Funktion, die ein neues Array erzeugt, welches nur jene Elemente des ursprünglichen Arrays enthält, für die eine übergebene Prädikatsfunktion `true` zurückgibt. Diese Methode implementiert das funktionale Konzept der *Filterung* und ist ein zentrales Werkzeug für deklarative Array-Transformationen. Die formale TypeScript Signatur sieht wie folgt aus:

```
1 filter(  
2   callbackfn: (value: number, index: number, array: number[]) => unknown,  
3   thisArg?: any  
4 ): number[]
```

Listing 23: Signatur *filter*

Die Callback-Funktion erhält automatisch drei Argumente:

1. `value`: Das aktuell verarbeitete Element
2. `index`: Der nullbasierte Index des Elements im Array
3. `array`: Das ursprüngliche Array, auf dem `filter` aufgerufen wurde

Im folgenden Beispiel werden nur gerade Zahlen aus dem Array extrahiert. Die Bedingung `n % 2 === 0` prüft, ob der Rest der Division durch 2 gleich 0 ist.

```

1      let K = [1, 2, 3, 4];
2      const result = K.filter((n, i, k) => {
3          console.log(n); // Aktuelles Element
4          console.log(i); // Index
5          console.log(k); // Ursprüngliches Array
6          return n % 2 === 0;
7      });
8      result; // Ergebnis: [2, 4]

```

Listing 24: Filtern gerader Zahlen

Die Ausführung lässt sich wie folgt visualisieren:

Element (n)	Operation (n % 2)	Test (=== 0)	Ergebnis
1	1 % 2 = 1	1 === 0 → false	Drop
2	2 % 2 = 0	0 === 0 → true	Keep
3	3 % 2 = 1	1 === 0 → false	Drop
4	4 % 2 = 0	0 === 0 → true	Keep

Erzeugung von Arrays mit `Array.from`

Neben der literalen Schreibweise bietet TypeScript die statische Methode `Array.from`, um Arrays dynamisch zu erzeugen. Diese Methode ist besonders nützlich, um Arrays einer bestimmten Länge zu initialisieren und gleichzeitig mit Werten zu befüllen. Sie akzeptiert ein *Array-ähnliches* Objekt (ein Objekt mit einer `length`-Eigenschaft) und eine optionale Mapping-Funktion. Die formale TypeScript Signatur sieht wie folgt aus:

```

1      static from<T, U>(
2          arrayLike: ArrayLike<T>,
3          mapfn: (element: T, index: number) => U
4      ): U[]

```

Listing 25: Signatur `from`

Im folgenden Beispiel wird ein Array erzeugt, das die Zahlenfolge von 1 bis 10 enthält:

```
1   Array.from({length: 10}, (_, i) => i + 1);  
2   // Ergebnis: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Listing 26: Erzeugung einer Zahlenfolge

Die Argumente werden dabei wie folgt interpretiert:

1. `{length: 10}`: Ein Objekt, das ein Array der Länge 10 simuliert. Da keine Elemente vorhanden sind, werden diese als **undefined** betrachtet.
2. `(_, i) => i + 1`: Die Mapping-Funktion. Der Unterstrich `_` ist eine Konvention für ein ignoriertes Argument (hier das **undefined**-Element). `i` ist der aktuelle Index (0 bis 9), der transformiert wird ($0 \rightarrow 1, \dots$).

Die Methode `map`

Die Methode `map` transformiert jedes Element eines Arrays mithilfe einer Callback-Funktion und gibt ein neues Array zurück, das die Ergebnisse dieser Transformation enthält. Die Länge des Arrays bleibt dabei unverändert.

```
1   const K = [1, 2, 3];  
2   K.map(n => n * n);  
3   // Ergebnis: [1, 4, 9]
```

Listing 27: Quadrieren von Zahlen mit `map`

Die Methode `reduce`

Die Methode `reduce` wird verwendet, um ein Array auf einen einzelnen Wert zu reduzieren (z. B. die Summe aller Elemente). Sie iteriert über das Array und führt einen Akkumulator mit, der den Zwischenstand der Berechnung speichert. Die formale TypeScript Signatur sieht wie folgt aus:

Schritt	a (Akkumulator)	b (Wert)	Operation (a + b)	Neu a
Start	0	—	—	0
1	0	1	0 + 1	1
2	1	2	1 + 2	3
3	3	3	3 + 3	6

```

1   reduce(
2   callbackfn: (accumulator: number, currentValue: number) => number,
3   initialValue: number
4   ): number

```

Listing 28: Signatur from

Die Callback-Funktion nimmt zwei Hauptargumente entgegen:

- **accumulator** (a): Der akkumulierte Wert (Startwert oder Ergebnis der vorherigen Iteration).
- **currentValue** (b): Das aktuelle Element des Arrays.

Das folgende Beispiel berechnet die Summe der Zahlen 1, 2 und 3:

```

1   [1, 2, 3].reduce((a, b) => a + b, 0);
2   // Ergebnis: 6

```

Listing 29: Summierung mit reduce

Der Ablauf der Berechnung (*Trace*) stellt sich wie folgt dar:

Array Destructuring

Ein leistungsfähiges Feature von TypeScript ist das *Array Destructuring*. Es ermöglicht das „Entpacken“ (*unpacking*) von Werten aus einem Array direkt in separate Variablen, anstatt mühsam über den Index (wie `arr[0]`, `arr[1]`) darauf zuzugreifen. Dies führt zu kompakterem und lesbarerem Code, insbesondere wenn Funktionen mehrere Werte in Form eines Arrays zurückgeben.

```
1   let a, b, rest;
2   [a, b] = [10, 20];
3   console.log(a); // Ausgabe: 10
4   console.log(b); // Ausgabe: 20
```

Listing 30: Einfaches Array Destructuring

Destructuring kann auch mit dem *Rest-Operator* (...) kombiniert werden, um die verbleibenden Elemente eines Arrays in einer neuen Array-Variable zu sammeln. Dies ist besonders nützlich, um den "Kopf" (Head) und den "Rest" (Tail) einer Liste zu trennen, ein gängiges Muster in der funktionalen Programmierung.

```
1   [a, b, ...rest] = [10, 20, 30, 40, 50];
2   console.log(rest);
3   // Ausgabe: [30, 40, 50]
```

Listing 31: Destructuring mit Rest-Operator

3.5 Tupel

Ein *Tupel* ist in TypeScript ein spezieller Array-Typ, der eine feste Anzahl von Elementen mit vorab definierten Datentypen besitzt. Im Gegensatz zu herkömmlichen Arrays, die meist homogen typisiert sind (z. B. `string[]`), erlauben Tupel die Speicherung einer heterogenen Sammlung von Werten in einer strikten Reihenfolge. Dieses Konzept eignet sich besonders, um zusammengehörige Daten zu gruppieren, ohne dafür eigens ein komplexes Objekt definieren zu müssen, beispielsweise für Koordinatenpaare `[x, y]` oder Rückgabewerte einer Funktion.

Im folgenden Beispiel wird ein Tupel-Typ `userProfile` definiert, der exakt drei Elemente in der Reihenfolge `number`, `string` und `boolean` erwartet:

```
1   let userProfile: [number, string, boolean]; // Typ-Definition
2   userProfile = [101, "Alice", true];
3   // Zuweisung: Reihenfolge und Typen müssen exakt übereinstimmen
```

Listing 32: Definition und Initialisierung eines Tupels

Der Zugriff auf die Elemente erfolgt analog zu Arrays über den Index-Operator:

```
1 console.log("Name:", userProfile[1]); // Ausgabe: Name: Alice
```

Listing 33: Zugriff auf Tupel-Elemente

3.6 Objekte

Während Arrays und Tupel Werte in einer geordneten Liste speichern, dienen *Objekte* der Speicherung von Schlüssel-Wert-Paaren (*Key-Value Pairs*). Ein Objekt ist eine Sammlung von Eigenschaften (*Properties*), wobei jede Eigenschaft einen Namen und einen zugehörigen Wert besitzt. In TypeScript werden Objekte mit geschweiften Klammern `{}` definiert.

Das nachstehende Beispiel zeigt ein `student`-Objekt mit grundlegenden Eigenschaften:

```
1 const student = {  
2     firstName: "Marvin",  
3     matriculationNumber: 123456,  
4     isEnrolled: true  
5 };
```

Listing 34: Objektdefinition

Zugriff auf Eigenschaften

Es existieren zwei gängige Notationen für den Zugriff auf Objekteigenschaften:

- **Punkt-Notation** (`obj.key`): Die Standard-Methode für den Zugriff, wenn der Schlüsselname bekannt und ein gültiger Bezeichner ist.
- **Klammer-Notation** (`obj["key"]`): Notwendig, wenn der Schlüssel in einer Variable gespeichert ist oder Sonderzeichen enthält.

```
1 console.log(student.firstName); // Punkt-Notation  
2 console.log(student["matriculationNumber"]); // Klammer-Notation
```

Listing 35: Zugriffsarten auf Objekteigenschaften

Objekte sind veränderlich (*mutable*), sodass Werte bestehender Eigenschaften nachträglich modifiziert werden können:

```
1      student.isEnrolled = false;
```

Listing 36: Modifikation von Eigenschaften

Verschachtelte Objekte

Objekte können beliebig verschachtelt werden, um komplexe Datenstrukturen abzubilden. Das Beispiel `university` demonstriert eine Hierarchie, in der die Eigenschaft `location` selbst ein Objekt ist und `departments` ein Array von Strings darstellt.

```
1      const university = {
2          name: "HTWG Konstanz",
3          location: {
4              street: "Alfred-Wachtel-Str. 8",
5              city: "Konstanz",
6              zipCode: 78462
7          },
8          departments: ["Computer Science", "Architecture",
9              "Mechanical Engineering"],
10         isPublic: true
11     };
```

Listing 37: Verschachtelte Objekte

Der Zugriff auf tiefere Ebenen erfolgt durch Verkettung des Punkt-Operators bzw. Kombination mit dem Array-Index:

```
1      // Zugriff auf verschachteltes Objekt
2      university.location.city; // Ausgabe: Konstanz
3
4      // Zugriff auf Array innerhalb eines Objekts
5      university.departments[1]; // Ausgabe: Architecture
```

Listing 38: Zugriff auf verschachtelte Daten

3.7 Boolesche Werte und Operatoren

Die Repräsentation von Wahrheitswerten in TypeScript erfolgt über den primitiven Datentyp `boolean`, der die Werte `true` und `false` annehmen kann. Diese Werte sind zentral

für die Steuerung des Programmflusses und logische Entscheidungen.

Logische Operatoren

TypeScript unterstützt die klassischen logischen Verknüpfungen Konjunktion (AND), Disjunktion (OR) und Negation (NOT) mittels der folgenden Operatoren:

```
1      true && false; // false
2      true || false; // true
3      !true; // false
```

Listing 39: Logische Verknüpfungen

Vergleichsoperatoren

Zur Erzeugung von Wahrheitswerten werden häufig numerische Vergleiche herangezogen. TypeScript bietet hierfür eine Reihe von Operatoren:

- `===`: Strikte Gleichheit (prüft Wert und Typ)
- `!==`: Strikte Ungleichheit
- `<`, `>`, `<=`, `>=`: Klassische Größenvergleiche

Ein wichtiger Unterschied zu Python besteht in der Verkettung von Vergleichen. Ein Ausdruck wie `1 < 2 < 3` wird in TypeScript nicht mathematisch ausgewertet. Stattdessen würde `1 < 2` zu `true` evaluiert, und anschließend versucht, `true < 3` zu berechnen (was durch Typ-Koerzition zu `1 < 3 → true` führen kann, aber oft nicht das Gewünschte ist). Korrekte mathematische Bereichsprüfungen müssen explizit mit `&&` verknüpft werden

Quantoren auf Arrays

TypeScript bietet Methoden auf Arrays, die den logischen Quantoren aus der Prädikatenlogik entsprechen:

1. **Allquantor** ($\forall$): Die Methode `every` prüft, ob *alle* Elemente eines Arrays eine Bedingung erfüllen.

```
1      const L = [2, 4, 0, 7];
2      // Sind alle Zahlen gerade? -> false, da 7 ungerade ist
3      L.every(x => x % 2 === 0);
```

2. **Existenzquantor** ($\exists$): Die Methode `some` prüft, ob *mindestens ein* Element die Bedingung erfüllt.

```
1      // Ist mindestens eine Zahl ungerade? -> true
2      L.some(x => x % 2 !== 0);
```

Diese Methoden ermöglichen eine deklarative und mathematisch fundierte Ausdrucksweise für Mengenoperationen direkt im Code.

3.8 Kontrollstrukturen

Kontrollstrukturen dienen dazu, den Ablauf eines Programms zu steuern. TypeScript bietet hierfür klassische Konstrukte, die syntaktisch stark an C, Java oder C++ angelehnt sind. Die grundlegende bedingte Ausführung erfolgt mittels `if`, `else if` und `else`. Im Gegensatz zu Python müssen die Bedingungen in runde Klammern `()` gesetzt und die Anweisungsblöcke in geschweifte Klammern `{}` eingeschlossen werden.

```
1      let x = 17;
2      if (x < 0) {
3          console.log('The number is negative!');
4      } else if (x === 0) {
5          console.log('The number is zero.');
```

```
6      } else {
7          console.log("It's more than one.");
8      }
```

Listing 40: If-Else Verzweigung

Alternativ bietet die `switch`-Anweisung eine strukturierte Möglichkeit, mehrere Bedingungen zu prüfen. Ein interessantes Muster in TypeScript ist das `switch(true)`-Konstrukt, das es erlaubt, komplexe Ausdrücke in den `case`-Zweigen auszuwerten:

```

1      switch (true) {
2          case (x < 0):
3              console.log('The number is negative!');
4              break;
5          case (x === 0):
6              console.log('The number is zero. ');
7              break;
8          default:
9              console.log("It's more than one.");
10     }

```

Listing 41: Switch-Statement als Alternative

Zur Iteration stellt TypeScript verschiedene Schleifenkonstrukte bereit:

- Die `for...of`-Schleife eignet sich ideal, um über die Elemente eines Arrays oder anderer iterierbarer Objekte zu laufen. Sie ist kompakter und weniger fehleranfällig als die klassische indexbasierte `for`-Schleife.
- Die `while`-Schleife führt einen Codeblock solange aus, wie eine definierte Laufbedingung erfüllt ist.

Anwendungsbeispiel: [Sieb des Eratosthenes](#)

Ein klassischer Algorithmus zur Bestimmung aller Primzahlen bis zu einer Grenze n ist das Sieb des Eratosthenes. Die Implementierung in TypeScript demonstriert das Zusammenspiel von Arrays, Schleifen und bedingten Anweisungen:

1. Initialisierung eines Arrays `primes`, wobei `primes[i] = i` gilt.
2. Iteration über alle Zahlen i und j , um Vielfache zu eliminieren (d. h. auf 0 zu setzen).
3. Filterung der verbleibenden Zahlen $\neq 0$.

Eine optimierte Version nutzt zudem `Math.sqrt(n)` als Obergrenze für die äußere Schleife und beginnt die innere Schleife bei $j = i$, um redundante Prüfungen zu vermeiden.


```

1      let n = 10000;
2      // Initialisiere Array mit Werten 0 bis n
3      let primes = Array.from({length: n + 1}, (_, i) => i);
4      let limit = Math.ceil(Math.sqrt(n));
5      for (let i = 2; i <= limit; i++) {
6          // Wenn i bereits markiert (0) ist, überspringen
7          if (primes[i] === 0) continue;
8          let j = i;
9          // Streiche alle Vielfachen von i (beginnend bei i*i)
10         while (i * j <= n) {
11             primes[i * j] = 0;
12             j++;
13         }
14     }
15     // Filtere alle Nicht-Null-Werte (Primzahlen)
16     let result = primes.filter(p => p !== 0 && p >= 2);

```

Listing 42: Optimiertes Sieb des Eratosthenes

3.9 Numerische Funktionen

TypeScript stellt, über das zugrundeliegende JavaScript, eine umfassende Sammlung mathematischer Standardfunktionen und Konstanten bereit, die im globalen `Math`-Objekt gekapselt sind. Dieses Objekt fungiert als Namensraum für statische Methoden und Eigenschaften. Zu den elementaren Funktionen gehören:

- **Wurzelberechnung:** `Math.sqrt(x)` berechnet die Quadratwurzel $\sqrt{x}$.
- **Betragsfunktion:** `Math.abs(x)` liefert den absoluten Wert $|x|$.
- **Potenzierung:** `Math.pow(base, exponent)` oder der Exponentialoperator `**`.

Rundungsfunktionen

Für die Umwandlung von Gleitkommazahlen in Ganzzahlen stehen drei spezifische Methoden zur Verfügung:

1. `Math.floor(x)`: Die *Gaußklammer unten* $\lfloor x \rfloor$. Sie rundet stets auf die nächstkleinere ganze Zahl ab.

$$\text{floor}(x) = \max\{n \in \mathbb{Z} \mid n \leq x\}$$

2. `Math.ceil(x)`: Die *Gaußklammer oben* $\lceil x \rceil$. Sie rundet stets auf die nächstgrößere ganze Zahl auf.

$$\text{ceil}(x) = \min\{n \in \mathbb{Z} \mid n \geq x\}$$

3. `Math.round(x)`: Kaufmännisches Runden zur nächsten ganzen Zahl. Hierbei wird $n.5$ auf $n + 1$ gerundet.

Trigonometrische Funktionen

Das `Math`-Objekt umfasst auch die Standard-Trigonometriefunktionen wie `Math.sin`, `Math.cos` und `Math.tan` sowie deren Arkusfunktionen (`asin`, `acos`, `atan`). Es ist zu beachten, dass diese Funktionen Winkel im Bogenmaß (Radiant) erwarten bzw. zurückgeben.

Exponential- und Logarithmusfunktionen

Die Exponentialfunktion e^x wird über `Math.exp(x)` berechnet. Das Inverse, der natürliche Logarithmus $\ln(x)$, wird in TypeScript (wie in vielen C-basierten Sprachen) als `Math.log(x)` bezeichnet.

Konstanten

Wichtige mathematische Konstanten sind direkt zugänglich:

- `Math.PI` für die Kreiszahl $\pi \approx 3.14159$
- `Math.E` für die Eulersche Zahl $e \approx 2.718$

4 Methodische Vorgehensweise

Dieses Kapitel dokumentiert den systematischen Übersetzungsprozess der Notebooks, angefangen beim grundlegenden Verständnis der Funktionen über die KI-gestützte Übersetzung bis hin zur Validierung der funktionsfähigen Endprodukte.

4.1 Projektmanagement mit GitHub Projects

Angeichts des erheblichen Umfangs von etwa 80 zu portierenden Jupyter Notebooks erfordert die Durchführung dieser Studienarbeit eine strukturierte Projektplanung und klare Aufgabenteilung. Um die Zusammenarbeit im Team zu gestalten und den Fortschritt transparent zu verfolgen, wird das integrierte Planungswerkzeug *GitHub Projects* eingesetzt. Zu Beginn des Projekts wurden alle Notebooks als einzelne Aufgaben (Tickets) im Backlog erfasst. Jedem Ticket wurde anschließend ein verantwortliches Teammitglied (*Assignee*) zugewiesen. Die visuelle Verwaltung dieser Aufgaben erfolgt über ein Kanban-Board, welches den Arbeitsfluss in verschiedene Phasen unterteilt (siehe Abbildung 1).

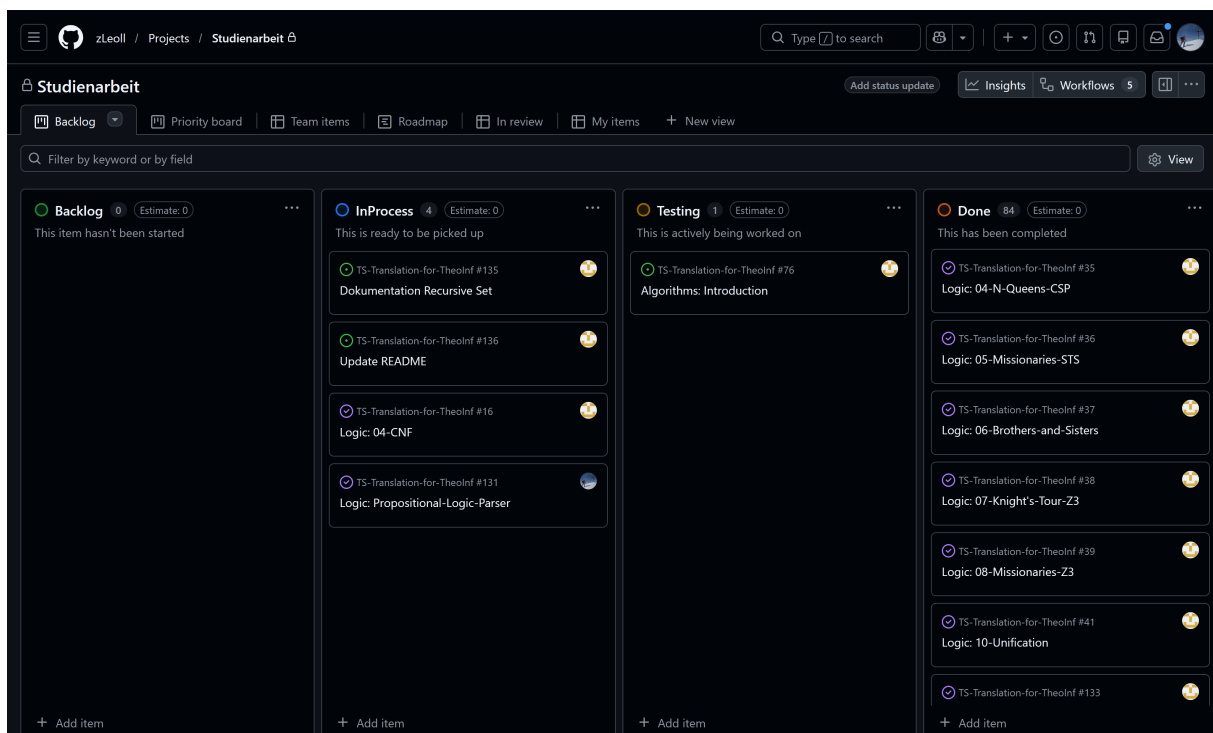


Abbildung 1: GitHub Projects Kanban-Board

Das in Abbildung 1 dargestellte Board gliedert den Workflow in vier Spalten:

- **Backlog:** Sammelbecken für alle noch nicht begonnenen Aufgaben.
- **InProcess:** Tickets, die aktuell von einem Teammitglied bearbeitet werden.
- **Testing:** Notebooks, deren Übersetzung abgeschlossen ist und die zur Überprüfung bereitstehen.
- **Done:** Erfolgreich validierte und abgeschlossene Notebooks.

Der Übersetzungsprozess folgt einem definierten Zyklus: Sobald die Bearbeitung eines Notebooks beginnt, wird das entsprechende Ticket vom *Backlog* in den Status *InProcess* verschoben. Nach Abschluss der Portierung und dem Push der Änderungen in das Versionskontrollsystem wechselt das Ticket in den Status *Testing*. Dieser Status signalisiert, dass das Notebook für ein Code-Review bereitsteht. Gemäß dem Vier-Augen-Prinzip überprüft nun ein anderes Teammitglied die Implementierung auf Korrektheit und Funktionalität. Erst nach erfolgreicher Validierung und gegebenenfalls notwendigen Korrekturschleifen wird das Ticket final in die Spalte *Done* überführt. Dieser strukturierte Prozess gewährleistet eine kontinuierliche Qualitätssicherung über den gesamten Projektverlauf hinweg.

Eine ergänzende methodische Maßnahme zur Vorbereitung der schriftlichen Ausarbeitung stellt die Nutzung von Labels dar. Um relevante Erkenntnisse und Code-Beispiele bereits während der Implementierungsphase systematisch zu erfassen, wurde das Label *writable* eingeführt. Notebooks, die spezifische Herausforderungen oder diskussionswürdige Code-Fragmente enthalten, wurden mit diesem Label markiert. Diese Verschlagwortung ermöglicht eine Filterung der Tickets im Nachgang und gewährleistet, dass wichtige Aspekte der Portierung gezielt in die Dokumentation der Studienarbeit einfließen können, ohne dass eine erneute, vollständige Sichtung aller Notebooks notwendig ist.

4.2 Der Übersetzungs-Workflow

Die Portierung der Jupyter Notebooks von Python nach TypeScript folgt einem systematischen und iterativen Prozess, der manuelle Analysen mit KI-gestützter Automatisierung kombiniert. Dieser hybride Ansatz gewährleistet sowohl Effizienz bei der Code-

Konvertierung als auch die qualitative Sicherung der Inhalte. Der Workflow gliedert sich in die folgenden Phasen:

4.2.1 Analyse und Vorbereitung

Zu Beginn wird das zu bearbeitende Python-Notebook aus dem Repository des Dozenten herunter geladen und lokal ausgeführt. Dieser Schritt dient dazu, ein initiales Verständnis für die implementierten Algorithmen und deren Laufzeitverhalten zu gewinnen. Die Analyse der Python-Ausgabe bildet die Referenzbasis (*Ground Truth*) für die spätere Validierung der TypeScript-Implementierung.

4.2.2 Hybride Code-Übersetzung

Der Kernprozess der Übersetzung erfolgt semi-automatisch unter Einbeziehung von KI-Modellen. Dabei werden einzelne Code-Snippets aus dem Python-Original extrahiert und mithilfe spezifischer Prompts, auf die im nachfolgenden Unterkapitel näher eingegangen wird, zur Übersetzung an das KI-Modell übergeben (siehe Abschnitt 4.3). Die generierten TypeScript-Fragmente werden anschließend manuell überprüft und optimiert. Hierbei liegt der Fokus auf zwei Aspekten:

1. **Funktionale Äquivalenz:** Der übersetzte Code muss dasselbe Verhalten wie das Original zeigen. Abweichungen, die durch sprachspezifische Unterschiede entstehen (z.B. Kontrollstrukturen wie `switch-case` vs. `if-elif-else`), werden explizit geprüft und bei Bedarf angepasst.
2. **Syntaktische Nähe:** Gemäß den Projektvorgaben soll die TypeScript-Implementierung strukturell so nah wie möglich an der Python-Vorlage bleiben, um die Nachvollziehbarkeit zu erhalten.

Bei Diskrepanzen zwischen den Sprachkonzepten wird ergänzend die offizielle TypeScript-Dokumentation [5] konsultiert, um eine geeignete Lösung zu finden.

4.2.3 Integration und Test

Nach der Übersetzung aller Code-Zellen erfolgt ein umfassender Integrationstest des gesamten Notebooks. Hierbei wird verifiziert, ob die einzelnen Komponenten korrekt in-

teragieren und das erwartete Gesamtergebnis liefern. Fehlerquellen, die durch die statische Typisierung von TypeScript aufgedeckt werden, sowie Laufzeitfehler werden in dieser Phase korrigiert. Sollte die Einbindung externer Bibliotheken notwendig sein, wird der entsprechende Installationsbefehl (`npm install ...`) in der zentralen Projektdokumentation (`README.md`) ergänzt, um die Reproduzierbarkeit der Entwicklungsumgebung sicherzustellen.

4.2.4 Adaption der Texte

Erst nach erfolgreicher Validierung der Code-Basis erfolgt die Anpassung der begleitenden Markdown-Texte. Da sich durch den Sprachwechsel oft auch die Semantik (z. B. Variablendeklaration, Typisierung) ändert, müssen die Erläuterungen entsprechend aktualisiert werden. Zudem werden neu eingeführte Hilfsfunktionen, die in der Python-Version nicht existent waren, aber für die TypeScript-Umsetzung notwendig sind, dokumentiert eingebettet.

4.2.5 Qualitätssicherung und Review

Den Abschluss des Workflows bildet ein zweistufiger Review-Prozess. Zunächst führt der Bearbeiter eine finale Selbstkontrolle von Code und Text durch. Anschließend wird das Notebook auf einen dedizierten Feature-Branch im Versionskontrollsystem gepusht. Dies löst den Review-Prozess durch den zweiten Entwickler aus (Vier-Augen-Prinzip). Der Reviewer prüft die Änderungen auf Korrektheit, Stil und Verständlichkeit. Erst nach Freigabe und eventueller Überarbeitung erfolgt der Merge in den Hauptzweig (*Main-Branch*) sowie der Abschluss des zugehörigen Tickets im Projektmanagement-Tool.

4.3 Einsatz von KI-gestützten Tools

Ein zentraler Bestandteil der methodischen Vorgehensweise ist der Einsatz generativer KI-Modelle zur Unterstützung des Übersetzungsprozesses. Da die manuelle Portierung von rund 80 Jupyter Notebooks extrem zeitaufwendig und fehleranfällig wäre, wurden moderne Large Language Models (LLMs) als Assistenzsysteme in den Workflow integriert.

4.3.1 Prompt-Engineering-Strategien für die Code-Übersetzung

Um konsistente und syntaktisch korrekte Ergebnisse zu erzielen, wurde ein standardisierter System-Prompt entwickelt, der den KI-Modellen als Kontextrahmen dient. Dieser Prompt adressiert spezifische Herausforderungen der Übersetzung von Python nach TypeScript, insbesondere im Hinblick auf das Typsystem und die modulare Struktur der Notebooks.

Der verwendete Basis-Prompt lautet:

Das Jupyter Notebook, welches ich dir hochgeladen habe, soll von Python nach TypeScript übersetzt werden. Ich werde dir in den folgenden Nachrichten einzelne Zellen des Notebooks geben und diese übersetzt du dann bitte einzeln in TypeScript. Falls in der Zelle eine Klasse definiert wird, denke daran, dass TypeScript typgecheckt ist und dass es alle Methoden kennen muss. Definiere deshalb in der gleichen Zelle mit der Klasse zusammen ein Interface, welches alle Methoden und Attribute der Klasse enthält, die später im Python-Notebook noch der Klasse angehängt werden (z.B. bei einer Klasse Trie mit: Trie.find = find; del find). Um in späteren Zellen diese Methoden der jeweiligen Klasse noch anzuhängen, verwende prototype (z.B.: Trie.prototype.find = function() ...). Beachte auch immer den Kontext mit Hilfe der Markdown-Zellen, die vor den Code-Zellen stehen. Falls Graphviz in Python vorkommt, verwende „@hpcc-js/wasm“ und für die Ausgabe von HTML-Inhalten verwende tslib mit display.html(...). Hier kommt die erste Zelle, die du übersetzen sollst: ...

Dieser Prompt wurde iterativ optimiert und adressiert mehrere kritische Aspekte:

- **Klassenstruktur und Typisierung:** Python erlaubt das dynamische Hinzufügen von Methoden zu Klassen zur Laufzeit (Monkey Patching [8]), was in den Algorithmen-Vorlesungen häufig genutzt wird. TypeScript als statisch typisierte Sprache verbietet dies standardmäßig. Der Prompt weist die KI explizit an, Interfaces und Prototypen-Manipulation (`prototype`) zu nutzen, um diese Dynamik typsicher nachzubilden.
- **Bibliotheksmapping:** Spezifische Python-Bibliotheken wie Graphviz werden direkt auf ihre TypeScript-Pendants (`@hpcc-js/wasm`) gemappt, um Halluzinationen bei der Bibliothekswahl zu vermeiden.

- **Kernel-Spezifikation:** Die Anweisung zur Nutzung von `display.html(...)` stellt sicher, dass die Ausgabe in Notebooks, die Graphviz verwenden, im Notebook-Kontext korrekt gerendert wird.

Ergänzend wurde in vielen Interaktionen der Constraint

”Verwende keinen any-Type, keine as-Casts und keine non-null assertion Operatoren”

hinzugefügt, um die KI zur Generierung von typsicherem Code zu zwingen und die Verwendung von „Escape Hatches“ (Umgehung des Typsystems) zu minimieren.

4.3.2 Verwendete Modelle und Evolution

Im Verlauf des Projekts wurden verschiedene KI-Modelle evaluiert und eingesetzt. Zu Beginn erfolgte die Übersetzung primär mit ChatGPT (Modell GPT-4/4o, später dann GPT-5/5.1). Während die grundlegende Syntaxübersetzung zufriedenstellend war, zeigten sich Schwächen bei komplexeren Kontextabhängigkeiten, etwa bei der Referenzierung von Code über mehrere Notebook-Zellen hinweg. Dies führte dazu, dass bei ca. 70% @zLeoll der generierten Fragmente manuelle Nachbesserungen erforderlich waren.

Aufgrund dieser Einschränkungen erfolgte ein Wechsel zur Plattform Perplexity AI. Diese zeichnete sich durch eine präzisere Kontextverarbeitung und eine höhere Code-Qualität aus, was den Nachbearbeitungsaufwand auf ca. 50% @zLeoll reduzierte. Ein wesentlicher Vorteil von Perplexity ist die Fähigkeit, dynamisch das geeignetste Modell für eine Anfrage auszuwählen. Insbesondere der Einsatz des Modells *Gemini 1.5 Pro auf Perplexity* (bzw. *Gemini 3 Pro* in der experimentellen Phase) erwies sich als besonders effektiv für die Übersetzung komplexer algorithmischer Logik.

4.3.3 Grenzen der KI-Unterstützung

Wir haben es auch probiert, ein ganzes Notebook hochzuladen, dabei ist aber die KI an die Grenzen gestoßen (lag wahrscheinlich an der Größe oder der Komplexität). Außerdem hat die KI trotzdem, wie oben bereits erwähnt, Fehler gemacht, welche man zuerst verstehen musste und dann manuell verbessern musste. Dazu war es besser, den Code aus dem Notebook einzeln zu übersetzen, um die Code-Snippets selbst zu verstehen und uns in das dementsprechende Thema wieder einzuarbeiten. Demnach war es zum Verständnis

und im Bezug auf der Gründlichkeit besser, die Codes aus dem Notebook einzeln der KI zu geben und zu übersetzen, statt ein gesamtes Notebook hoch zu laden. Trotz des Einsatzes leistungsfähiger Modelle stieß auch die KI-Unterstützung an systembedingte Grenzen:

- **Halluzinationen bei Nischen-Bibliotheken:** Bei selten verwendeten TypeScript-Bibliotheken neigten die Modelle dazu, nicht existente Methoden zu erfinden oder APIs von ähnlichen JavaScript-Bibliotheken zu vermischen.
- **Kontextlänge:** Der Versuch, ganze Notebooks in einem Schritt zu übersetzen, führte häufig zu unvollständigen Ergebnissen oder dem Verlust von Details. Die abschnittsweise Übersetzung (SZelle für Zelle”), wie im Prompt definiert, erwies sich als robusterer Ansatz, auch wenn sie den manuellen Koordinationsaufwand erhöhte.

5 Case Studies

@Leoll Im Verlauf der Studienarbeit sind wir auf mehrere komplexe Problemstellungen/Hürden gestoßen. Im folgenden haben wir uns Fälle aus Notebooks herausgesucht, welche wir zum Verständnis genauer aufzeigen möchten.

5.1 CNF

Die Umwandlung der Python-Notebooks für den Propositional Logic Parser und die CNF-Konvertierung in TypeScript erforderte fundamentale strukturelle Anpassungen, die primär aus den unterschiedlichen Typsystemen und Datenstrukturen der beiden Sprachen resultieren. Im Folgenden werden die wesentlichen Änderungen und deren Notwendigkeit detailliert erläutert.

5.1.1 Type-System und strukturelle Typdefinitionen

Der gravierendste Unterschied zwischen beiden Implementierungen liegt im Typsystem. Während Python ein dynamisches Typsystem mit optionalen Type Hints verwendet, verlangt TypeScript explizite Typdeklarationen, die zur Compile-Zeit geprüft werden.

Rekursive Formeldefinition:

In der Python-Version wurde die Formelstruktur wie folgt definiert:

```
1 Formula = TypeVar('Formula')
2 Formula = str | tuple[Formula, ...]
```

Listing 43: Python: Generische rekursive Formeldefinition

Diese Definition ist bewusst generisch (**@TobiNight** hier generisch erklären + Quelle) gehalten und erlaubt beliebige Tupel-Strukturen. TypeScript erfordert hingegen eine präzise strukturelle Definition:

```

1 export type Formula = Variable | ['⊤' | '⊥'] | ['¬', Formula] |
2   ['↔' | '→' | '∧' | '∨', Formula, Formula];

```

Listing 44: TypeScript: Strukturell präzise Formeldefinition

Diese explizite Definition modelliert exakt die Grammatik der Aussagenlogik als algebraischen Datentyp. Jede Variante des Union Type entspricht einer Produktionsregel:

- `Variable` repräsentiert atomare Propositionen $p, q, r, \dots$
- `['⊤' | '⊥']` kodiert die nullstelligen Junktoren
- `['¬', Formula]` repräsentiert die Negation als einstelligen Operator
- `['↔' | '→' | '∧' | '∨', Formula, Formula]` modelliert alle zweistelligen Junktoren als Tupel mit genau zwei Teilformeln

Die Typdefinition erzwingt strukturelle Wohlgeformtheit: Ein Ausdruck wie `['∧', 'p']` (Konjunktion mit nur einem Operanden) oder `['¬', '∨', 'p', 'q']` (syntaktisch falsche Verschachtelung) wird zur Compile-Zeit abgelehnt. Damit wird die syntaktische Korrektheit von Formeln nicht erst zur Laufzeit durch Parser-Fehler erkannt, sondern bereits durch das Typsystem garantiert. Dies ist besonders wichtig für die nachfolgenden Transformationsfunktionen, die auf der strukturellen Integrität der Formelbäume aufbauen – eine falsch strukturierte Formel könnte zu Laufzeitfehlern in Pattern-Matching-Ausdrücken oder zu logisch inkorrekten Transformationen führen.

5.1.2 Typprogression durch Transformationsschritte

Eine besondere Stärke der TypeScript-Implementation liegt in der expliziten Modellierung der Transformationsschritte durch progressive Typdefinitionen:

```

1 // Nach Eliminierung der Bikonditionale
2 export type Formula1 = Variable | ['T' | '⊥'] |
3   ['¬', Formula1] | ['→' | '∧' | '∨', Formula1, Formula1];
4
5 // Nach Eliminierung der Konditionale
6 export type Formula2 = Variable | ['T' | '⊥'] |
7   ['¬', Formula2] | ['∧' | '∨', Formula2, Formula2];

```

Listing 45: Progressive Typdefinitionen für Transformationsschritte

Diese Typen dokumentieren präzise, welche Operatoren nach jedem Transformationsschritt noch in der Formel vorhanden sein können. Die Funktion `eliminateBiconditional` nimmt ein `Formula` entgegen und gibt ein `Formula1` zurück – der Typ garantiert, dass keine 'biconditional'-Operatoren mehr vorhanden sind. Entsprechend eliminiert `eliminateConditional` alle 'conditional'-Operatoren und gibt ein `Formula2` zurück.

Der finale Typ NNF (Negation Normal Form) kodiert zusätzlich die Einschränkung, dass Negationen nur noch auf atomare Variablen angewendet werden:

```

1 type NNF = Variable | ['T'] | ['⊥'] |
2   ['¬', Variable] | ['∧' | '∨', NNF, NNF];

```

Listing 46: NNF-Typ mit eingeschränkter Negation

Diese schrittweise Verfeinerung der Typen macht die Algorithmen selbstdokumentierend und verhindert Fehler durch falsche Annahmen über die Formelstruktur.

5.1.3 RecursiveSet: Notwendigkeit und Implementierung

Die wichtigste strukturelle Änderung betrifft die Repräsentation von Klauseln und CNF-Formeln. Python verwendet native `frozenset`-Objekte:

```
1 Variable = str
2 Literal = Variable | tuple[str, Variable]
3 Clause = frozenset[Literal]
4 CNF = set[Clause]
```

Listing 47: Python: Verwendung von frozenset für Klauseln

JavaScript bzw. TypeScript verfügt jedoch nicht über eine native Set-Implementation, die strukturelle Gleichheit (value equality) für verschachtelte Strukturen unterstützt. Das native `Set` in JavaScript verwendet Referenzgleichheit, was zu folgenden Problemen führt:

```
1 // JavaScript-Problem: Referenzgleichheit
2 const set1 = new Set();
3 const literal1 = ['¬', 'p'];
4 const literal2 = ['¬', 'p'];
5 set1.add(literal1);
6 set1.add(literal2);
7 console.log(set1.size); // Ausgabe: 2 (erwartet: 1)
8
9 // literal1 !== literal2 (verschiedene Referenzen)
10 // obwohl strukturell identisch
```

Listing 48: Problem der Referenzgleichheit in JavaScript

Dieses Problem ist besonders kritisch für den Davis-Putnam-Algorithmus und andere Resolutionsverfahren, die auf der eindeutigen Repräsentation von Klauseln und Literalen basieren. Ohne korrekte Duplikaterkennung würde der Algorithmus:

- Identische Klauseln mehrfach speichern und verarbeiten
- Redundante Resolutionsschritte durchführen
- Falsche Ergebnisse bei der Erfüllbarkeitsprüfung liefern
- Exponentiell mehr Speicher verbrauchen

Die Lösung ist die Verwendung der `recursive-set` Library, die strukturelle Gleichheit implementiert:

```

1  import { RecursiveSet, Tuple } from 'recursive-set';
2
3  export type Literal = Variable | Tuple<['¬', Variable]>;
4  export type Clause = RecursiveSet;
5  export type CNF = RecursiveSet;
6
7  // Korrekte Duplikaterkennung
8  const clause = new RecursiveSet();
9  clause.add(new Tuple('¬', 'p'));
10 clause.add(new Tuple('¬', 'p'));
11 console.log(clause.size); // Ausgabe: 1 (korrekt)

```

Listing 49: TypeScript: RecursiveSet mit struktureller Gleichheit

Die Tuple-Klasse aus derselben Library wird verwendet, um Literale wie $\neg p$ zu repräsentieren. Sie garantiert ebenfalls strukturelle Gleichheit und ist hashbar, sodass sie als Element in `RecursiveSet` verwendet werden kann.

5.1.4 Literal-Repräsentation und Komplementbildung

Ein weiterer wichtiger Unterschied liegt in der Repräsentation von Literalen. In Python:

```

1  # Python: Literale als Union von str und tuple
2  Literal = Variable | tuple[str, Variable]
3
4  # Beispiele:
5  positiv = 'p'                                # Positive Literal
6  negativ = ('¬', 'p')                         # Negative Literal

```

Listing 50: Python: Literal-Repräsentation

In TypeScript musste die Struktur angepasst werden, um mit `RecursiveSet` kompatibel zu sein:

```

1 // TypeScript: Literale mit Tuple-Wrapper
2 export type Literal = Variable | Tuple<['¬', Variable]>;
3
4 // Hilfsfunktion für Komplementbildung
5 function getComplement(l: Literal): Literal {
6     if (typeof l === 'string') {
7         return new Tuple('¬', l);
8     } else {
9         return l.get(1);
10    }
11 }

```

Listing 51: TypeScript: Literal-Repräsentation mit Tuple

Die Funktion `getComplement` ist essentiell für die Triviality-Prüfung von Klauseln. Eine Klausel ist trivial (immer wahr), wenn sie sowohl ein Literal p als auch sein Komplement $\neg p$ enthält:

```

1     export function isTrivial(clause: Clause): boolean {
2         for (const lit of clause) {
3             const comp = getComplement(lit);
4             // RecursiveSet.has() verwendet strukturelle Gleichheit
5             if (clause.has(comp)) {
6                 return true;
7             }
8         }
9         return false;
10    }

```

Listing 52: Triviality-Prüfung mit struktureller Gleichheit

Diese Implementation funktioniert korrekt nur durch die strukturelle Gleichheit von `RecursiveSet`. Mit einem normalen JavaScript-Set würde die Prüfung `clause.has(comp)` fehlschlagen, selbst wenn das Komplement strukturell vorhanden ist.

5.2 Pattern Matching und Type Guards

Python 3.10+ unterstützt strukturelles Pattern Matching mit `match`-Statements, die besonders elegant für die Verarbeitung von Formelstrukturen sind:

```

1  def eliminateBiconditional(f: Formula) -> Formula:
2      match f:
3          case str(p):
4              return p
5          case ('↔', g, h):
6              return eliminateBiconditional(('^', ('→', g, h), ('→', h, g)))
7          case ('¬', g):
8              return ('¬', eliminateBiconditional(g))
9          case (op, g, h):
10             return (op, eliminateBiconditional(g), eliminateBiconditional(h))

```

Listing 53: Python: Pattern Matching mit match-Statement

TypeScript hingegen unterstützt kein direktes Pattern Matching, daher werden `switch`-Statements mit Type Guards und Destructuring verwendet:

```

1  export function eliminateBiconditional(f: Formula): Formula1 {
2      if (typeof f === 'string') {
3          return f;
4      }
5      const [op] = f;
6      switch (op) {
7          case '↔': {
8              const [, g, h] = f;
9              return eliminateBiconditional(['^', ['→', g, h], ['→', h, g]]);
10         }
11         case '¬': {
12             const [, g] = f;
13             return ['¬', eliminateBiconditional(g)];
14         }
15         // ... weitere Cases
16     }
17 }

```

Listing 54: TypeScript: Switch-Statement mit Type Guards

Der Type Guard `typeof f === 'string'` ermöglicht es dem Compiler, in den nachfolgenden Zweigen zu erkennen, dass `f` eine der Array-Varianten des Types sein muss: `['⊤' | '⊥']` | `['¬', Formula]` | `['↔' | '→' | '^' | '∨', Formula, Formula]`. Im

case ' $\leftrightarrow$ '-Zweig verfeinert der Compiler den Typ weiter auf `[' $\leftrightarrow$ ', Formula, Formula]`, sodass die Destructuring-Syntax `const [, g, h] = f` die beiden Operanden als Typ `Formula` extrahiert, während das erste Element (der Operator ' $\leftrightarrow$ ') übersprungen wird.

5.3 CNF-Konstruktion und Set-Operationen

Die eigentliche CNF-Konstruktion zeigt deutlich die Unterschiede in der Set-Manipulation. Python verwendet Set Comprehensions:

```
1 def cnf(f: Formula) -> CNF:
2     match f:
3         case str(p):
4             return { frozenset({p}) }
5         case ('^', g, h):
6             return cnf(g) | cnf(h)
7         case ('v', g, h):
8             return { k1 | k2 for k1 in cnf(g) for k2 in cnf(h) }
9         # ...
```

Listing 55: Python: CNF-Konstruktion mit Set Comprehensions

Die distributive Regel für die Disjunktion ($g \vee h$) erzeugt alle Kombinationen von Klauseln aus g und h und vereinigt diese paarweise. In Python ist dies durch die prägnante Set Comprehension ausdrückbar.

TypeScript benötigt explizite Schleifen und `RecursiveSet`-Operationen:

```

1  export function cnf(f: NNF): CNF {
2      if (typeof f === 'string') {
3          const clause = new RecursiveSet(f);
4          return new RecursiveSet(clause);
5      }
6      switch (f[0]) {
7          case '^': {
8              const [, g, h] = f;
9              const left = cnf(g);
10             const right = cnf(h);
11             return left.union(right);
12         }
13         case 'v': {
14             const [, g, h] = f;
15             const left = cnf(g);
16             const right = cnf(h);
17             const result = new RecursiveSet();
18             for (const c1 of left) {
19                 for (const c2 of right) {
20                     const unionClause = c1.union(c2);
21                     result.add(unionClause);
22                 }
23             }
24             return result;
25         }
26         // ...
27     }
28 }

```

Listing 56: TypeScript: CNF-Konstruktion mit expliziten Schleifen

Die verschachtelte Schleife für die Disjunktion iteriert über alle Klauseln beider Teilformeln und vereinigt sie. `RecursiveSet` garantiert hier automatisch, dass keine Duplikate entstehen – sowohl auf Klausel- als auch auf Literal-Ebene. Dies ist essentiell für die Effizienz nachfolgender Algorithmen.

5.4 Praktische Auswirkungen für Davis-Putnam

Die Verwendung von `RecursiveSet` ist nicht nur eine technische Notwendigkeit, sondern hat direkte Auswirkungen auf die Korrektheit und Effizienz des Davis-Putnam-Algorithmus:

1. **Unit-Propagation:** Wenn eine Unit-Klausel $\{l\}$ existiert, müssen alle Klauseln, die $\neg l$ enthalten, entfernt werden. Ohne strukturelle Gleichheit würden identische Klauseln mehrfach vorkommen und die Propagation wäre inkonsistent.
2. **Pure-Literal-Elimination:** Die Identifikation reiner Literale (die in keiner Klausel negiert vorkommen) erfordert korrekte Duplikaterkennung. Ansonsten würde ein Literal fälschlicherweise als gemischt klassifiziert.
3. **Resolution:** Bei der Resolution zweier Klauseln über ein Literal l entsteht die Resolvente durch Vereinigung unter Ausschluss von l und $\neg l$. Duplikate müssen hier automatisch eliminiert werden.
4. **Speichereffizienz:** In realen Anwendungen können CNF-Formeln Tausende von Klauseln enthalten. Ohne Duplikaterkennung würde der Speicherverbrauch exponentiell wachsen.

Ein Beispiel verdeutlicht das Problem:

```

1 // Formel:  $(p \vee q) \wedge (p \vee q)$ 
2 // Ohne RecursiveSet:  $\{\{p, q\}, \{p, q\}\}$  - zwei Klauseln
3 // Mit RecursiveSet:  $\{\{p, q\}\}$  - eine Klausel
4
5 const formula = ['^', ['v', 'p', 'q'], ['v', 'p', 'q']];
6 const result = normalize(formula);
7 // result.size === 1 (korrekt)
8
9 // Ohne RecursiveSet würden identische Klauseln
10 // mehrfach im Davis-Putnam verarbeitet

```

Listing 57: Duplikateliminierung in der Praxis

5.5 Zusammenfassung der Kernänderungen

Die Migration von Python zu TypeScript erforderte folgende fundamentale Anpassungen:

- **Explizite Typdefinitionen:** TypeScript verlangt strukturell präzise Union Types für Formeln, die zur Compile-Zeit validiert werden.
- **Progressive Typen:** Die Transformationsschritte werden durch separate Typen (Formula1, Formula2, NNF) dokumentiert und erzwungen.
- **RecursiveSet statt frozenset:** Zwingend notwendig für strukturelle Gleichheit bei verschachtelten Strukturen wie Klauseln und CNF-Mengen.
- **Tuple-Wrapper:** Negative Literale müssen als `Tuple<['¬', Variable]>` statt einfache Arrays repräsentiert werden, um Hashbarkeit und strukturelle Gleichheit zu garantieren.
- **Explizite Schleifen:** Set Comprehensions müssen durch verschachtelte `for`-Schleifen mit `RecursiveSet`-Operationen ersetzt werden.
- **Switch statt Match:** Pattern Matching wird durch `switch`-Statements mit Type Guards und Destructuring simuliert.

Alle diese Änderungen waren nicht optionale Verbesserungen, sondern essentielle Anpassungen an die Eigenheiten der TypeScript/JavaScript-Plattform. Insbesondere

`RecursiveSet` ist unverzichtbar für die korrekte Funktionsweise des Davis-Putnam-Algorithmus und aller darauf aufbauenden SAT-Solver, da nur so die eindeutige Repräsentation von Klauseln und die automatische Duplikateliminierung garantiert werden kann.

5.6 Graphviz

...

5.7 Wiederverwendung von Notebooks

...

5.8 Recursive Set

Erläuterung und Nutzen

6 Evaluierung und Ergebnisse

In diesem Kapitel werden wir auf den Erfolg der Übersetzung vergleichen.

6.1 Laufzeit

...

6.2 Lesbarkeit

...

6.3 Effizienz

-i z.B. 8 Damen Problem -i Code-Snipppets (so, dass sie relevant sind); allgemeine Suchalgorithmen)

```
1
2      // Ein Interface definieren
3      interface User {
4          id: number;
5          name: string;
6      }
7
8      // Eine Funktion mit Typisierung
9      function greet(user: User): string {
10         return `Hallo, ${user.name}!`;
11     }
12
13     const myUser: User = { id: 1, name: "Max" };
14     console.log(greet(myUser));
```

Listing 58: TypeScript: Beispiel mit Interface und Funktion

7 Fazit und Ausblick

Beispiel FOL-Parser viel zu großer Umfang

Literaturverzeichnis

- [1] *Algoithms Repository*. Website. URL: <https://github.com/karlstroetmann/Algorithms>. (Zugriff am 13. Oktober 2025).
- [2] *Logic Repository*. Website. URL: <https://github.com/karlstroetmann/Logic>. (Zugriff am 13. Oktober 2025).
- [3] *Teaching and Learning with Jupyter*. Website. URL: <https://jupyter4edu.github.io/jupyter-edu-book/>. (Zugriff am 19. Januar 2026).
- [4] *TypeScript*. Website. URL: <https://en.wikipedia.org/wiki/TypeScript>. (Zugriff am 25. Januar 2026).
- [5] *TypeScript Documentation*. Website. URL: <https://www.typescriptlang.org/docs/>. (Zugriff am 25. Januar 2026).
- [6] *TypeScript Repository*. Website. URL: <https://github.com/Microsoft/TypeScript?tab=readme-ov-file>. (Zugriff am 25. Januar 2026).
- [7] *Understanding TypeScript*. Website. URL: https://link.springer.com/chapter/10.1007/978-3-662-44202-9_11. (Zugriff am 19. Januar 2026).
- [8] *What is Monkey Patching*. Website. URL: <https://stackoverflow.com/questions/5626193/what-is-monkey-patching>. (Zugriff am 15. Februar 2026).